

BULLSEYE

AI-Powered Investment & Trading Assistant

Comprehensive Project Report

Full-Stack Fintech Platform with Real-Time Market Data, ML Predictions & AI Insights

Author	Shrish (B tech in CSE AI & ML)
Date	December 2025 – May 2026
Project Type	Full-stack Fintech Platform with Real-Time Market Data, ML Prediction &AI Insights
Contacts	shrishdas444@gmail.com , +91-9162271138

1. Executive Summary

Bullseye is a full-stack, India-first fintech platform that unifies live NSE market data, candlestick charting, AI-powered chat, machine-learning price predictions, portfolio management, risk analysis, and financial news sentiment into a single intelligent interface. It is engineered with a production-style async Python backend (FastAPI), a reactive TypeScript/React frontend, two complementary ML models (XGBoost and LSTM), and Google Gemini as its conversational AI engine.

Bullseye is not a demo. It is a production-grade system with real-time WebSocket streaming, JWT authentication, async database sessions, dynamic NSE symbol resolution, on-demand ML model training, and a modular provider abstraction layer.

Attribute	Detail
Platform Type	Full-Stack Fintech SaaS (Web)
Target Market	Indian equity investors and active traders using NSE instruments
Backend	FastAPI + Python + async SQLAlchemy + SQLite
Frontend	React 18 + TypeScript + Vite + TailwindCSS + Framer Motion
AI Engine	Google Gemini 2.5 Flash + VADER Sentiment + Sentence Transformers (RAG-ready)
ML Models	XGBoost (primary) + LSTM/Keras (sequence learning) with auto-training pipeline
Market Data	Upstox API (live quotes, historical candles, NSE instrument registry)
Live Updates	WebSocket streaming with REST fallback
Auth	JWT + bcrypt, protected routes, centralized token management
Key Features	Live charts, AI chat, ML predictions, portfolio tracking, risk analysis, alerts, news sentiment, dashboard

2. Motivation & Problem Statement

2.1 Why Bullseye Was Built

Indian retail investors typically rely on a fragmented ecosystem of tools: one app for charts, another for news, a spreadsheet for portfolio tracking, and various forums for analysis. This fragmentation slows decision-making, increases cognitive load, and leaves beginners without contextual explanations of what they are seeing.

Bullseye was built to consolidate these experiences into one intelligent platform that not only shows data, but explains it using AI.

2.2 Core Problems Addressed

Problem	Impact on Users	Bullseye's Solution
Information Fragmentation	Traders switch between 4–6 tools per session, wasting time and missing signals	Unified dashboard: live prices, charts, news, AI chat, and portfolio in one place
Explainability Gap	Raw RSI / MACD values are meaningless to many investors	AI-powered indicator explanations using Gemini, triggered with one click
Prediction Blindspot	No accessible ML-driven direction signals for retail traders	XGBoost + LSTM models auto-trained per symbol, giving directional predictions with confidence scores
Portfolio Blindness	Manual spreadsheets with no live P/L enrichment	Live-price-enriched portfolio with sector allocation, P/L per holding, and CSV import
Risk Unawareness	Investors cannot quantify concentration or volatility risk	Full risk analysis: Sharpe, Beta, VaR 95%, HHI, drawdown, and AI narrative
Sentiment Vacuum	News is consumed without structured market sentiment signals	VADER-scored news feed with per-article sentiment and overall market tone

3. System Architecture & Data Pipeline

3.1 High-Level Architecture

Bullseye follows a layered, service-oriented architecture with a clear separation between data ingestion, business logic, AI/ML inference, and presentation. The diagram below represents the runtime data flow.

SYSTEM ARCHITECTURE OVERVIEW
[EXTERNAL DATA SOURCES] Upstox API (live quotes, historical candles, NSE instrument registry) NewsData.io (breaking financial news) Google Gemini AI (LLM reasoning & explanation)
[BACKEND -- FastAPI / Python] Instrument Registry -> Loads & caches 10,000+ NSE symbols at startup Market Provider Router -> Resolves symbol to Upstox instrument key WebSocket -> Streams live LTP to browser clients every second REST APIs -> /market, /portfolio, /risk, /alerts, /dashboard, /predict, /chat, /news Auth Layer -> JWT issuance & verification, bcrypt password hashing Async SQLAlchemy -> SQLite for users, holdings, alerts, prices
[M L PIPELINE] Raw OHLCV Candles -> Feature Engineering (RSI, EMA, MACD, ATR, volume spike, momentum) MLDatasetBuilder -> Creates target labels (UP / SIDEWAYS / DOWN) via quantile method XGBoostPredictor -> Tabular classifier, auto-trains on first request per symbol LSTMPredictor -> Sequence model (sequence_length=20), complements XGBoost PredictionService -> Caches loaded models + scalars, returns confidence + signals
[AI LAYER] Gemini 2.5 Flash -> Chat Q&A, indicator explanations, portfolio insights, risk narrative VADER Sentiment -> Scores each news article and computes overall market sentiment Sentence Transformers -> Embeds user queries for RAG-ready architecture
[FRONTEND -- React + TypeScript] Pages: Dashboard, Market, Portfolio, Risk, Predictions, Alerts, News, Chat, Settings Charts: Lightweight TradingView candlesticks + Recharts analytics State: Zustand (auth) + TanStack React Query (server data + caching)

Realtime: WebSocket client for live price stream with REST fallback

Figure 1. Bullseye end-to-end system architecture and data pipeline.

3.2 Request Lifecycle

A typical market data request flows as follows:

- User selects a symbol (e.g., RELIANCE) on the Market page.
- Frontend opens a WebSocket connection to /ws/market/RELIANCE and fires a REST request to /api/market/candles/RELIANCE.
- Backend's Market Provider Router calls `get_instrument_key('RELIANCE')` on the in-memory NSE instrument registry.
- The resolved instrument key (e.g., NSE_EQ|INE002A01018) is passed to `UpstoxProvider.fetch_candles()`.
- Technical indicators (RSI, EMA, SMA, MACD) are computed server-side and attached to each candle object.
- The enriched candle array is returned to the frontend and rendered in the candlestick chart with overlays.
- Simultaneously, WebSocket pushes live LTP updates every second to the price ticker.
- The user can click 'AI Explain Indicators' — the latest indicator values are sent to Gemini, which returns a plain-English explanation.

4. Feature Deep-Dive

4.1 Live Dashboard

The Dashboard is the command center of Bullseye. It aggregates data from six backend subsystems into a single API call (/api/dashboard/summary) and renders a real-time investment overview.

Dashboard Component	Data Source	What It Shows
Portfolio Value Card	portfolio_crud + live Upstox prices	Total invested, current value, overall P/L in INR and % with a 30-day sparkline
Total P/L Card	Holdings + live prices	Absolute and percentage gain/loss with a visual bar indicator
Alerts Card	alerts table	Active alert count, total triggers, quick link to alert management
Risk Level Card	Risk snapshot (HHI, flags)	Overall risk rating (Low / Moderate / High) and concentration score
Market Pulse Panel	Upstox daily candles for watchlist	Price, 1-day and 5-day change, mini sparkline, and RSI badge for 6 key stocks
Holdings P&L Bar Chart	Portfolio holdings + prices	Recharts bar chart of P/L% per holding, color-coded green/red
Asset Allocation Donut	Portfolio holdings	SVG donut chart showing allocation by asset type (stock, ETF, crypto, mutual fund)
Recent Alerts Panel	triggered_alerts table	Last 3 triggered alerts with symbol, message, and timestamp
Quick Navigation Strip	Static routes	One-click links to Portfolio, Risk, Predictions, and Alerts pages

The dashboard refreshes every 60 seconds automatically. Each section degrades gracefully when live price data is unavailable — invested value is displayed instead of current value, and null-safe rendering prevents blank cards.

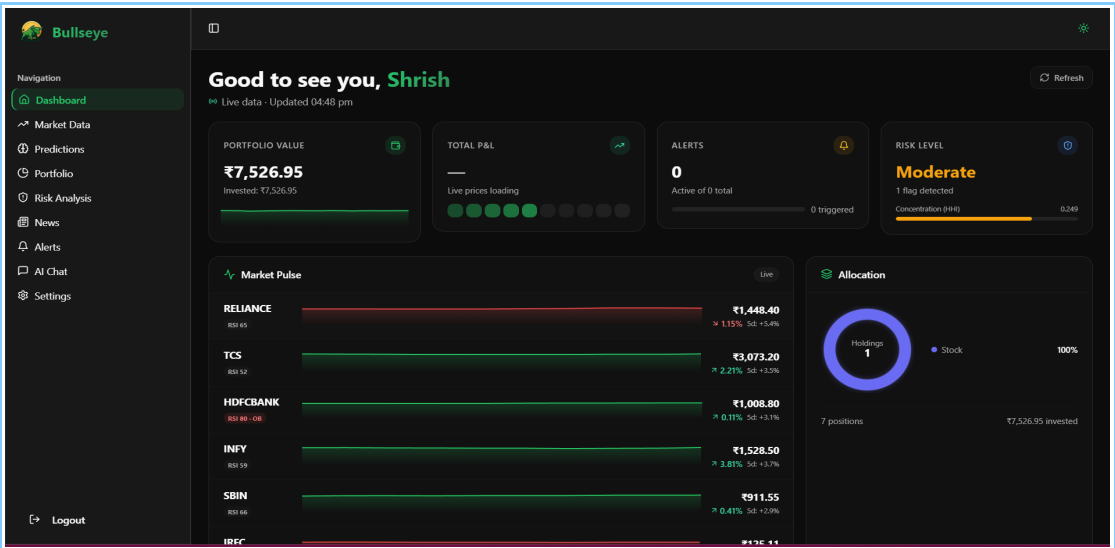


Figure 1: Live Dashboard

4.2 Real-Time Market Data & Charting

The Market page provides professional-grade candlestick charting with live price streaming and one-click AI indicator explanations.

- **Symbol Search:** Free-text input with debouncing (600ms) prevents excessive API calls as the user types.
- **Candlestick Chart:** Rendered using a charting library. Each candle includes OHLCV data plus overlaid RSI, SMA, EMA, and MACD computed server-side.
- **Live Price (WebSocket):** A persistent WebSocket connection streams the Last Traded Price every second. If the WebSocket is unavailable, the system falls back to a REST quote call.
- **Timeframe Selector:** Supports 1m, 5m, 15m, 60m, and Daily intervals. Resolution is mapped to Upstox-compatible intervals on the backend.
- **AI Explain Indicators:** The latest RSI, SMA, and EMA values are sent to Gemini with structured prompts. Gemini returns a 3-point explanation (RSI interpretation, SMA vs EMA relationship, overall market sentiment) displayed below the chart.

Technical Insight: All indicator computation (RSI, EMA, SMA, MACD) is performed server-side in indicators.py and attached to each candle. This keeps the frontend lightweight and ensures consistent calculations across all users.



Figure 2: Real time market trend

4.3 AI Chat Assistant

The Chat page provides a conversational interface powered by Google Gemini 2.5 Flash. The assistant is contextualized as 'Bullseye' - an AI investment advisor for Indian markets.

- **RAG-Ready Architecture:** User questions are first embedded using Sentence Transformers. The EmbeddingsStore performs a vector similarity search and retrieves relevant document chunks. This context is prepended to the Gemini prompt.
- **Gemini Integration:** The LLMClient wraps the Google GenAI SDK. Google Search Retrieval is enabled, allowing Gemini to fetch live information when needed.
- **System Prompt Engineering:** The system prompt instructs the model to be educational, avoid guaranteed financial advice, and use retrieved context when available.
- **Quick Prompts:** Pre-built prompt chips ('Analyze TCS stock', 'Explain portfolio volatility') lower the barrier to entry for new users.
- **Persistent Conversation:** Full message history is maintained in React state and displayed in a chat-bubble UI with timestamps.

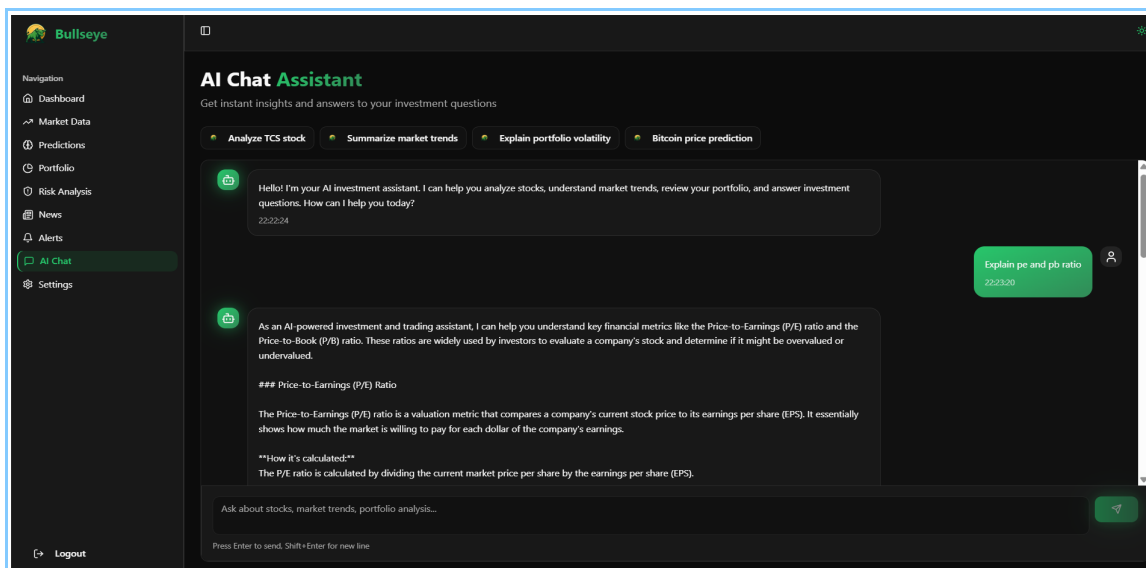


Figure 3: AI Chat Assistant Powered by Google Gemini

4.4 AI Stock Predictions

The Predictions page is the most technically sophisticated feature. It provides machine-learning-driven directional forecasts (UP / SIDEWAYS / DOWN) for any NSE symbol, with confidence scores and explainable signals.

ML Pipeline Architecture

Stage	Component	Detail
1. Data Fetching	UpstoxProvider.fetch_candles()	200 daily candles fetched for training, 120 for inference
2. Indicator Calculation	indicators.py	RSI(14), SMA(14), EMA(9), EMA(21), MACD computed and attached to DataFrame
3. Feature Engineering	PriceActionFeatures	Returns, volatility, ATR, trend structure (higher highs/lows), momentum, volume spikes, price-

		volume correlation
4. Signal Features	TechnicalSignals	RSI overbought/oversold/momentum, EMA crossover direction and distance, MACD histogram and crossovers
5. Target Creation	TargetBuilder	Looks 10 candles ahead; uses quantile-based labeling for balanced UP/SIDEWAYS/DOWN classes
6. Dataset Split	MLDatasetBuilder	Time-series-aware split (no shuffle): 70% train, 15% validation, 15% test
7. Scaling	StandardScaler	Fit on training data only; scaler saved as scaler_{SYMBOL}.pkl for inference reuse
8. XGBoost Training	XGBoostPredictor	Multi-class softmax, 200 estimators, early stopping on validation loss
9. LSTM Training	LSTMPredictor	2-layer LSTM [64, 32], sequence_length=20, dropout=0.2, EarlyStopping
10. Auto-Training	PredictionService._auto_train()	Triggered on first prediction request for an unseen symbol; models cached in memory after first load
11. Inference	PredictionService.predict()	Scales features, runs model, returns label + confidence + expected move + signals

Model Selection Rationale

- **Chosen for its strong performance on structured tabular data, fast inference, built-in feature importance, and robustness to outliers. It is the recommended model for most use cases.** XGBoost (Primary):
- **Chosen for its ability to capture temporal dependencies across sequences of candles. Useful when sufficient historical data exists and sequential patterns matter. Slower to train but can capture momentum shifts XGBoost misses.** LSTM (Supplementary):

Prediction Output

For each prediction request, the API returns:

- symbol, prediction (UP/SIDEWAYS/DOWN), confidence (0–100%), expected_move (+/- %)
- current_price, signals (human-readable list of triggered indicators), model_used, timestamp

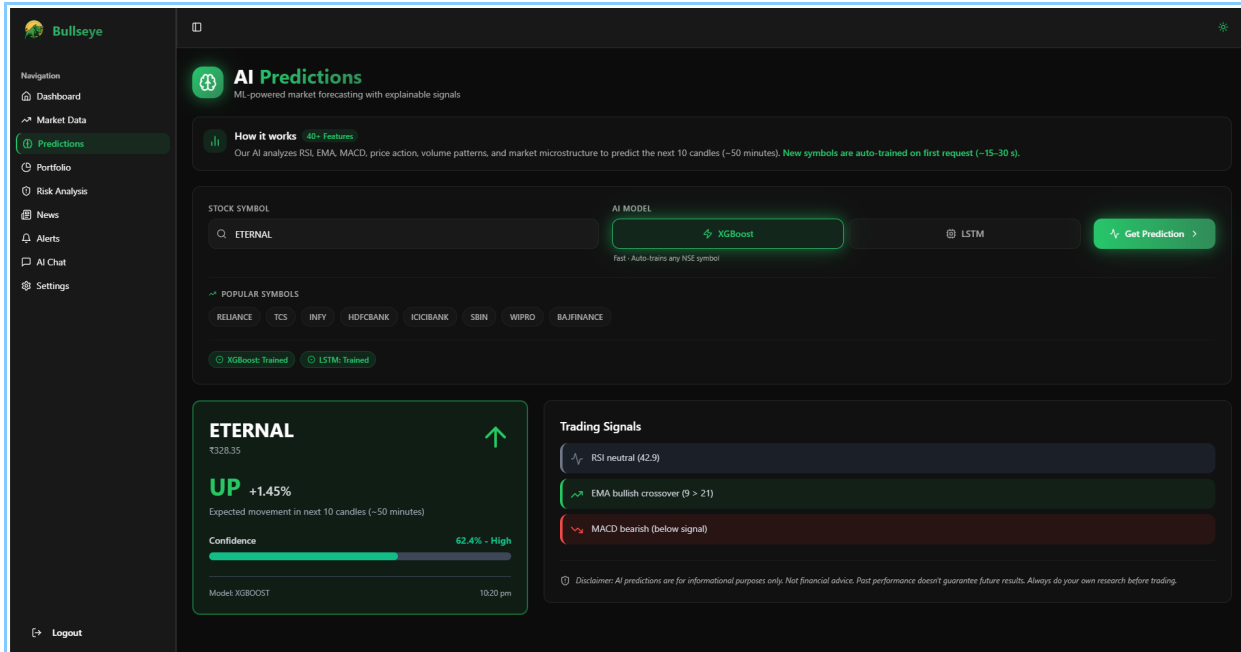


Figure 4: (a) Stock Prediction using XGBoost and LSTM (upward trend)

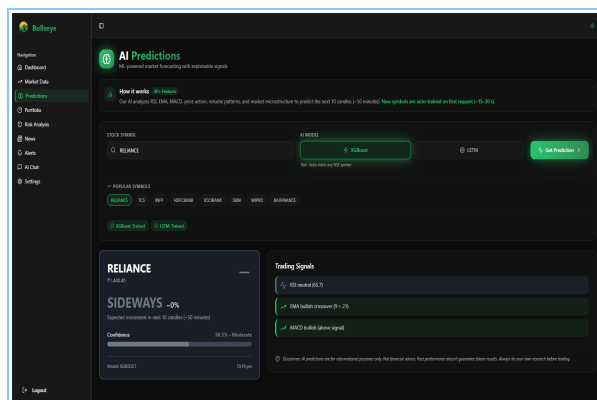


Figure 5: (b) Sideways Trend

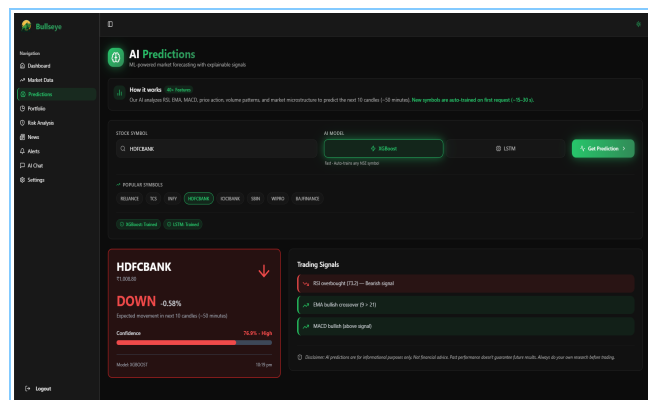


Figure 6: (c) Downward trend

4.5 Portfolio Management

The Portfolio page provides a comprehensive investment tracking system with live price enrichment, sector-level analytics, AI-powered insights, and bulk import capability.

Feature	How It Works
Holdings List	Displays all holdings with live P/L. Prices are fetched concurrently using <code>asyncio.gather()</code> for speed. Each holding shows symbol, quantity, buy price, current price, P/L in INR and %, and portfolio allocation %.
Manual Add Holding	Form with symbol, quantity, buy price, buy date, and asset type. Asset type is auto-detected from symbol patterns (crypto, ETF, mutual fund, or stock).
CSV / Excel Import	Drag-and-drop or click to upload .csv / .xlsx / .xls. Backend uses pandas to parse, normalizes column names, validates each row, and reports per-row errors. Duplicate symbols within the upload are

	skipped.
Sector Allocation Pie Chart	Recharts donut chart with sector assignment mapped from symbol (Banking, Technology, Energy, Pharma, FMCG, Auto, Metals, ETF/Funds, Others).
Portfolio Summary	Aggregates total invested, total current value, total P/L, and holdings count across all positions.
AI Portfolio Insights	Sends a structured text summary of all holdings with P/L to Gemini. Gemini responds with an ~300-word analysis covering Overview, Top Winners/Losers, Concentration Risk, and Recommendations.

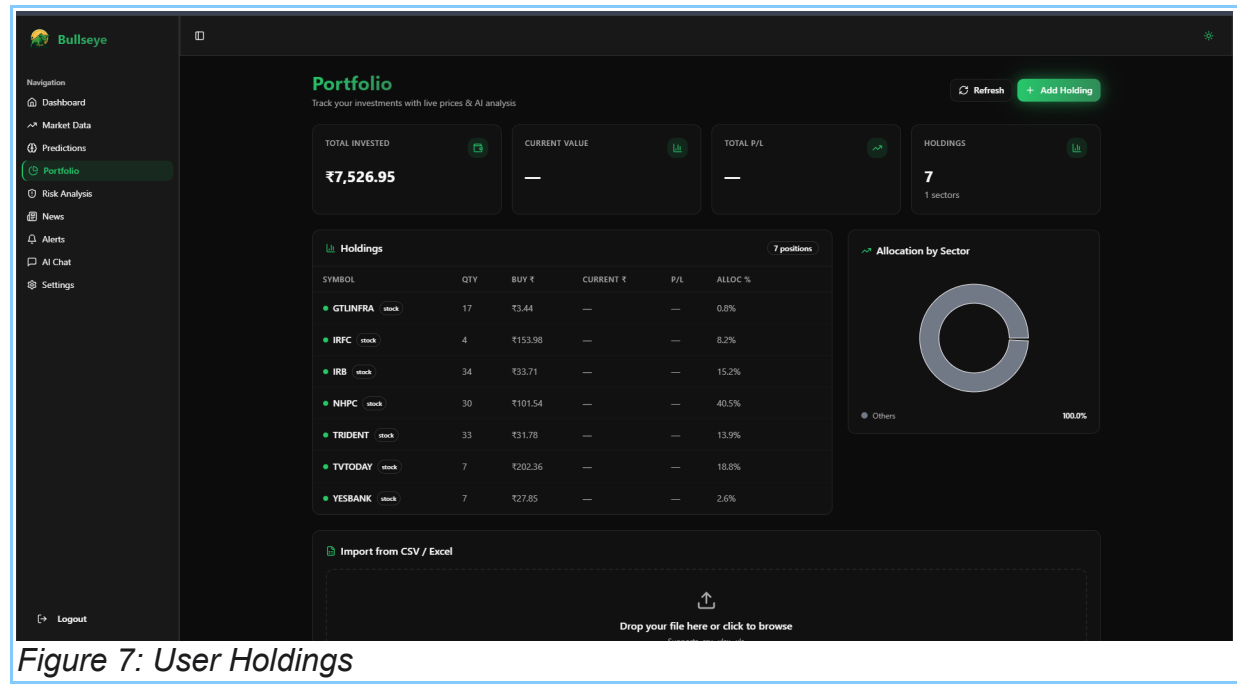


Figure 7: User Holdings

4.6 Risk Analysis

The Risk Analysis page quantifies portfolio danger across multiple dimensions and provides a stress-testing tool for simulating market crashes.

Metric	Calculation	Interpretation
Portfolio Volatility	Weighted avg of annualised std-dev of daily returns (x sqrt(252))	Overall risk level (Low <15%, Moderate <25%, High <40%)
Portfolio Beta	OLS regression of stock returns vs NIFTY/RELIANCE proxy returns	Sensitivity to broad market moves (Beta>1 = amplified moves)
Sharpe Ratio	((mean daily return - risk_free/252) / std) x sqrt(252)	Risk-adjusted return (>1 = good, >2 = excellent)
VaR 95%	Historical 5th percentile of daily returns	Maximum expected daily loss at 95% confidence
HHI Concentration	Sum of squared portfolio weights	>0.25 signals dangerous concentration in a single position
Max Drawdown	Peak-to-trough decline in closing prices over 90 days	Worst historical loss from a peak
RSI Flags	RSI > 75 or < 25 on any holding	Signals potential reversal risk or oversold opportunity

The Stress Test module accepts a market drop percentage (1–50%) and computes, per holding, the Beta-adjusted expected price drop and portfolio P/L impact. It presents a comparison of current value vs. stressed value with an impact percentage.

An AI Risk Assessment section sends all per-holding metrics to Gemini, which returns a structured narrative (Risk Overview | Key Risks | Recommendations).

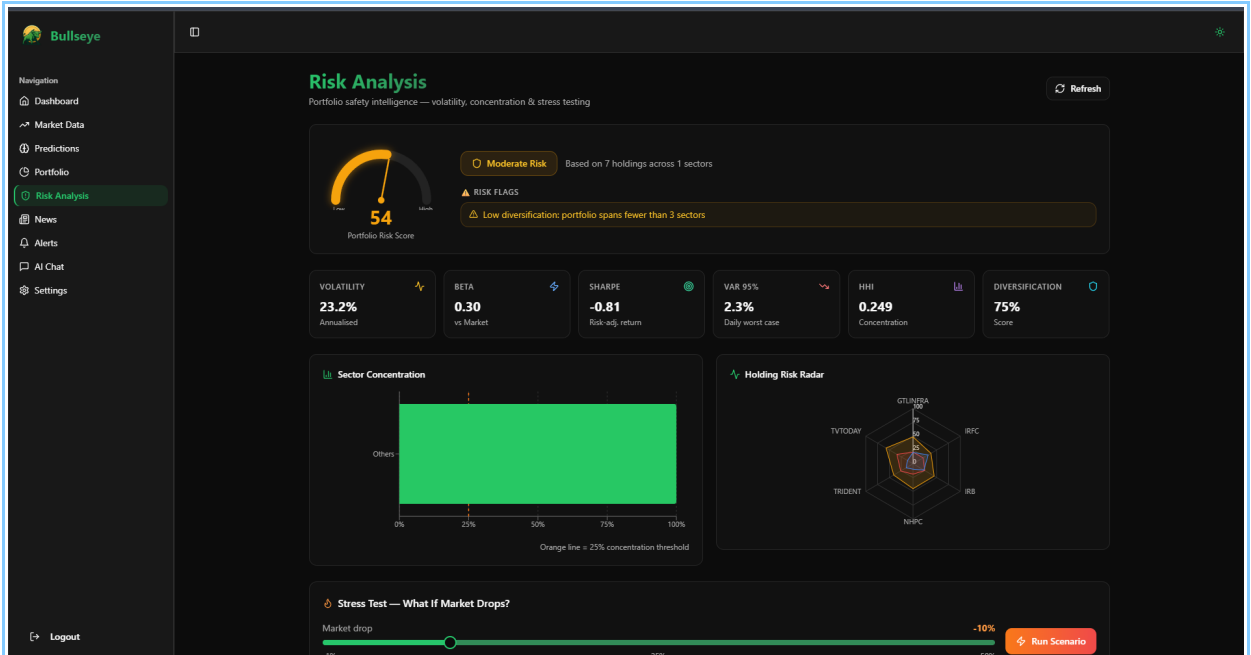


Figure 8: Risk management

4.7 Smart Alerts System

The Alerts page enables users to configure real-time market monitoring conditions that are evaluated on demand or on a schedule.

Alert Type	Trigger Condition	Market Data Used
price_above	LTP >= threshold	Upstox live quote (fetch_quote)
price_below	LTP <= threshold	Upstox live quote (fetch_quote)
rsi_overbought	RSI(14) >= threshold (default 70)	Last 60 daily candles, RSI computed server-side
rsi_oversold	RSI(14) <= threshold (default 30)	Last 60 daily candles, RSI computed server-side
volume_spike	Current volume >= threshold x 20-day avg volume	Last 25 daily candles, volume ratio calculated

Alert state is persisted in the database. Each trigger is recorded in triggered_alerts with the current value and a human-readable message. The 'Check Now' button evaluates all active alerts concurrently using asyncio.gather() and returns a summary of triggered conditions.

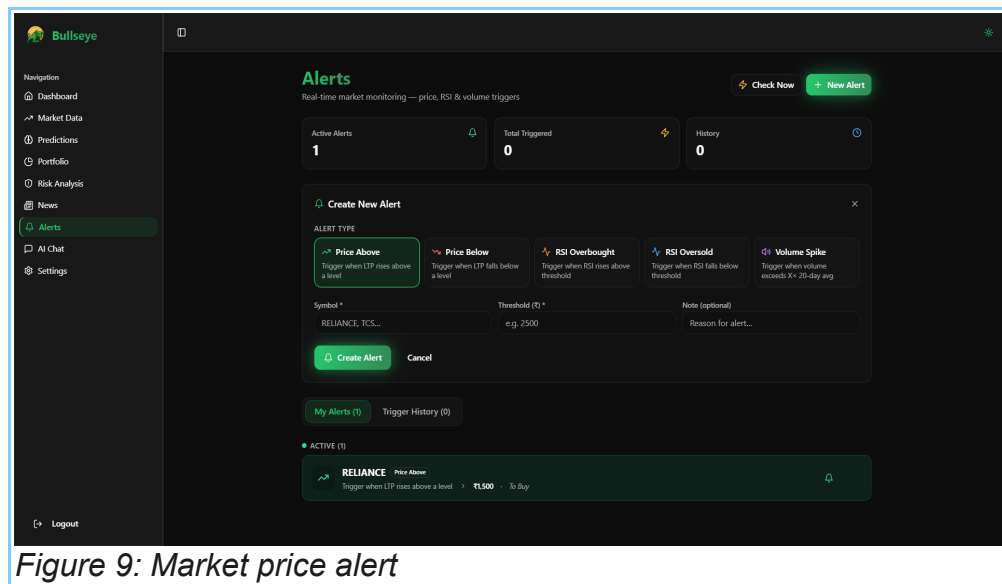


Figure 9: Market price alert

4.8 Financial News & Sentiment

The News page fetches breaking Indian market news from NewsData.io and enriches each article with VADER sentiment analysis.

- Articles are filtered by Indian business news keywords (NSE, BSE, RBI, Sensex, Nifty, Indian market).
- Each article receives a VADER compound sentiment score and a label (Positive / Negative / Neutral).
- An overall Market Sentiment banner aggregates the mean score across all articles and labels the market Bullish / Bearish / Neutral.
- Articles are displayed in a card feed with title, description, source, publication time, image, and sentiment badge.

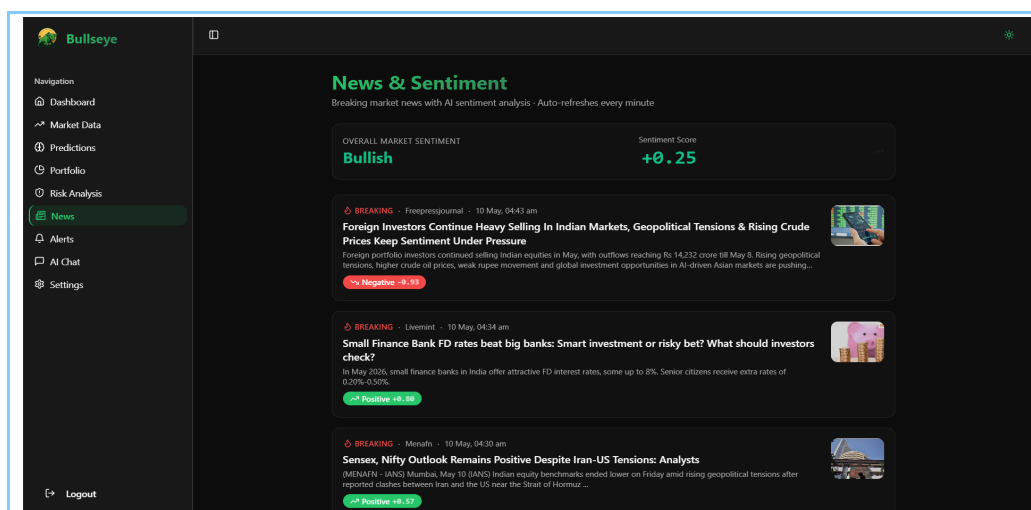


Figure 10: National and International Financial News & Sentiment

5. Technology Stack — Roles & Rationale

Layer	Technology	Role in Bullseye	Why Chosen
Frontend Framework	React 18 + TypeScript + Vite	Builds all UI pages, components, and routing	Type safety reduces runtime bugs; Vite provides fast HMR for development speed
Styling	Tailwind CSS + Framer Motion	Glassmorphism design system, animations, responsive layout	Utility-first CSS for rapid iteration; Framer Motion for smooth page transitions
State Management	Zustand + TanStack React Query	Auth/session state (Zustand); server data cache with refetch intervals (Query)	Lightweight and composable; React Query eliminates manual loading/error states
Charting	Recharts + Lightweight Charts (TradingView)	Candlestick charts with indicator overlays; portfolio donut, bar, and area charts	TradingView provides professional OHLCV charting; Recharts handles analytics views
Backend Framework	FastAPI + Python 3.11 + Uvicorn	REST API endpoints, WebSocket server, dependency injection, request validation	Async-native framework; fastest Python API throughput; auto-generates OpenAPI docs
Database ORM	SQLAlchemy async + Pydantic v2	Async DB sessions, model definitions, request/response schema validation	Clean ORM with async support; Pydantic v2 for strict data contracts
Database	SQLite (aiosqlite)	Stores users, portfolio holdings, alerts, triggered events, prices	Zero-configuration for development; swap to PostgreSQL for production with no ORM changes
Authentication	JWT (python-jose) + bcrypt	Token issuance on login, protected route middleware, password hashing	Stateless and horizontally scalable; bcrypt is industry standard for password storage
Market Data	Upstox API	NSE instrument registry (10,000+ symbols), live LTP quotes, historical candles	Only major Indian broker API with a comprehensive REST and WebSocket API for NSE equity data
Live Streaming	WebSocket (FastAPI native)	Real-time LTP updates to browser clients every second	Lower latency than polling; FastAPI supports WebSocket natively without extra dependencies
LLM / AI Chat	Google Gemini 2.5 Flash	Conversational Q&A, indicator explanation, portfolio analysis, risk narrative	Best-in-class speed-to-quality ratio; native Google Search Retrieval for live data grounding
Embeddings	Sentence Transformers (all-MiniLM-L6-v2)	Embeds user queries for semantic similarity search (RAG-ready)	Lightweight open-source model; runs locally without API costs
Sentiment NLP	VADER	Scores financial news headlines and descriptions	Specifically calibrated for social and short-text

			sentiment; no GPU required; runs synchronously
ML - Tabular	XGBoost	Price direction classification from engineered features	State-of-the-art gradient boosting for tabular data; fast training and inference; interpretable feature importance
ML - Sequence	TensorFlow / Keras LSTM	Sequence-based price pattern learning	Captures temporal dependencies in candle sequences that XGBoost cannot model natively
Feature Engineering	Scikit-learn + TA (ta library)	StandardScaler, ATR, feature pipeline	StandardScaler for normalization; ta library for ATR computation
News API	NewsData.io	Fetches Indian business news with category and country filtering	Free tier supports up to 10 articles per request with good Indian coverage
Async Task Queue	Celery + Redis (planned)	Background model training, price ingestion tasks	Standard Python async task processing for heavy background jobs
Deployment	Render (backend) + Vercel(frontend) + Docker	Production hosting, containerization, CI/CD-ready	Render provides free-tier Python deployments; Docker ensures environment parity

6. Machine Learning Pipeline

6.1 Feature Engineering (40+ Features)

The feature set is organized into two classes:

PriceActionFeatures

Feature Group	Features Generated
Returns	1-candle return, 5-candle return, log return
Volatility	Rolling std-dev, Average True Range (ATR), normalized ATR %
Trend Structure	Rolling 20-period high/low, higher high indicator, lower low indicator, trend strength (% from rolling mean)
Momentum	Rate of Change (ROC) at 5 and 10 periods, 5-period price momentum
Volume	20-period volume MA, volume spike binary flag (>1.5x avg), volume trend (5-period pct change), price-volume correlation

TechnicalSignals

Signal Group	Features Generated
RSI Signals	rsi_oversold (RSI<30), rsi_overbought (RSI>70), rsi_neutral, rsi_momentum (diff)
EMA Signals	ema_bullish_cross (EMA9>EMA21), ema_fresh_cross_up, ema_fresh_cross_down, ema_distance %
MACD Signals	macd_bullish (above signal), macd_cross_up, macd_histogram

6.2 Target Variable Construction

Targets are generated by the TargetBuilder class using a quantile-based 3-class labeling scheme:

- Lookahead: 10 candles (approximately 50 minutes on 5-minute data, or 10 trading days on daily data)
- Method: `pd.qcut(future_return, q=3)` — divides returns into three equal-sized quantile buckets
- Labels: -1 (DOWN), 0 (SIDEWAYS), 1 (UP) — balanced classes avoid class imbalance issues that plague threshold-based labeling

6.3 Model Evaluation Metrics

Both models are evaluated using the ModelEvaluator class, which reports:

- Classification: Accuracy, Precision, Recall, F1-Score (weighted), per-class confusion matrix
- Trading-specific: Directional Accuracy (UP/DOWN only), Win Rate, Average Return/Trade, Total Return, Sharpe Ratio, UP/DOWN Prediction Precision

6.4 Auto-Training & Caching

The PredictionService handles the full model lifecycle:

- On first request for symbol X: checks for saved model files (`xgboost_X.json`, `lstm_X.keras`, `scaler_X.pkl`).

- If missing: triggers `_auto_train(X)`, which runs the full training pipeline (data fetch, feature engineering, train/val/test split, both models, save artifacts).
- Model and scaler objects are cached in-memory after first load, making subsequent predictions instantaneous.
- The prediction endpoint also generates a training report (JSON) per symbol for auditability.

7. API Design — Endpoint Reference

Module	Endpoint	Method	Description
Auth	/api/auth/signup	POST	Register a new user; returns UserOut
Auth	/api/auth/login	POST	Login with email/password; returns JWT access token
Market	/api/market/quote/{symbol}	GET	Fetch live LTP for any NSE symbol
Market	/api/market/candles/{symbol}	GET	Historical + intraday OHLCV candles with indicators
Market	/api/market/prices/{symbol}	GET	Recent stored prices from DB
WebSocket	/ws/market/{symbol}	WS	Live price stream (pushes LTP every second)
Chat	/api/chat/query	POST	AI chat with RAG context; returns answer + source IDs
Chat	/api/chat/explain-indicators	POST	AI explanation of RSI, SMA, EMA for a symbol
Portfolio	/api/portfolio/holdings	GET	All holdings with live price enrichment
Portfolio	/api/portfolio/holdings	POST	Add a new holding manually
Portfolio	/api/portfolio/holdings/{id}	DELETE	Remove a holding
Portfolio	/api/portfolio/summary	GET	Aggregate P/L summary
Portfolio	/api/portfolio/import	POST	Bulk import from CSV/XLSX file
Portfolio	/api/portfolio/ai-insights	GET	Gemini analysis of portfolio holdings
Risk	/api/risk/analysis	GET	Full portfolio risk report (volatility, beta, Sharpe, VaR, HHI)
Risk	/api/risk/volatility/{symbol}	GET	Single-symbol volatility and technical metrics
Risk	/api/risk/scenario	POST	Stress test — what-if market drop simulation
Alerts	/api/alerts/	GET/POST	List or create alerts
Alerts	/api/alerts/{id}	PUT/DELETE	Update or delete an alert
Alerts	/api/alerts/check	GET	Evaluate all active alerts against live market data
Alerts	/api/alerts/triggered	GET	Recent alert trigger history
Predictions	/api/predict/{symbol}	GET	AI prediction with confidence, signals, and expected move
Predictions	/api/predict/{symbol}/models	GET	Check which models are trained for a symbol
News	/api/news/breaking	GET	Latest Indian market news with VADER sentiment scores
Dashboard	/api/dashboard/summary	GET	Unified dashboard data (portfolio + alerts + market + risk)
Health	/api/health/ping	GET	Server liveness check

8. Engineering Challenges & Solutions

The following table documents the most significant production-style challenges encountered during development. Solving these problems is what differentiates Bullseye from a tutorial project.

Challenge	Root Cause	Solution Applied
LSTM sequence failure on small data	Insufficient candles for <code>sequence_length=20</code> ; empty sequence arrays caused crashes	Dynamic <code>sequence_length</code> reduction, safe fallback to uniform probability arrays, data window increased to 120+ days for inference
XGBoost <code>predict_proba</code> returned wrong shape	<code>set_param()</code> mutation permanently changed model objective to <code>softprob</code> , breaking next <code>predict()</code> call	Replaced with <code>output_margin=True</code> approach: get raw margins, apply manual softmax — eliminates objective mutation entirely
Upstox 401 on intraday candles	Free-tier Upstox credentials do not support intraday historical endpoints	Switched all ML training and inference to daily candles (<code>resolution='D'</code>); auto-detects subscription limitation
NSE instrument key resolution failing	Raw tickers (RELIANCE) rejected; Upstox requires format <code>NSE_EQ INE002A01018</code>	Built <code>InstrumentRegistry</code> that downloads, caches (24h), and indexes 10,000+ NSE symbols at startup
Async SQLAlchemy <code>IllegalStateChangeError</code>	Sessions were manually closed while transactions were still active	Proper dependency-injected async session lifecycle using <code>contextmanager</code> pattern in <code>get_db()</code>
CORS errors blocking all browser requests	<code>CORSMiddleware</code> wired after other middleware in incorrect startup order	Centralized all middleware registration in <code>main.py</code> lifespan context; CORS set before other middleware
JWT tokens not attached to requests	Each page fetched tokens independently with inconsistent storage keys	Created centralized <code>apiFetch()</code> helper that always reads token from <code>localStorage</code> and injects <code>Authorization</code> header
Model file not found for new symbols	System designed to require pre-trained artifacts; no fallback for unknown symbols	Implemented <code>_auto_train()</code> in <code>PredictionService</code> — triggered automatically, trains both models, saves artifacts, invalidates cache
Date range errors from Upstox	Backend computed future dates; Upstox rejected <code>to_date > today</code>	Added <code>UPSTOX_MAX_DATE</code> guard and capped to <code>_date</code> to <code>min(today, UPSTOX_MAX_DATE)</code>
Poor prediction accuracy (~23%) on small datasets	Training on 30-60 days of data with 40+ features leads to overfitting	Expanded to 200 days for training, improved feature engineering, and added quantile-balanced target labels
Pydantic v1/v2 dependency conflicts	Mixed pydantic v1 (validators) and v2 (<code>model_validator</code>) syntax across files	Full migration to Pydantic v2 style throughout; pinned pydantic-settings separately
TradingView Lightweight Charts v5 API breaking change	Old <code>addSeries()</code> API removed; chart rendering failed silently after library upgrade	Migrated to new v5 <code>addSeries()</code> pattern; added timestamp normalization for backend candle format

9. Key Architectural Design Decisions

9.1 Server-Side Indicator Computation

All technical indicators (RSI, EMA, SMA, MACD) are computed in the backend's `indicators.py` module before sending data to the frontend. This ensures:

- Identical calculations across all browser clients and API consumers.
- The frontend remains lightweight — it renders, not computes.
- ML features in the prediction pipeline use the same indicator logic as the charting API.

9.2 Provider Abstraction Layer

A `MarketProvider` abstract base class (`base.py`) defines the interface for `fetch_quote()` and `fetch_candles()`. The `ProviderRouter` resolves any symbol and returns the correct provider instance. This means:

- Swapping from Upstox to Zerodha or any other broker requires only a new `Provider` class.
- The rest of the codebase (APIs, ML pipeline, `WebSocket`) remains unchanged.
- `Finnhub` is included as a fallback for international symbols.

9.3 Quantile-Based ML Target Labels

Rather than fixed thresholds ($+0.5\% = \text{UP}$), Bullseye uses `pd.qcut()` to divide the future return distribution into three equal-frequency buckets. This guarantees balanced training classes regardless of market conditions or the symbol's historical volatility range.

9.4 Time-Series-Aware Train/Test Split

The dataset builder explicitly avoids `sklearn`'s `train_test_split` (which shuffles data). Instead, it splits chronologically: the first 70% of records go to training, the next 15% to validation, and the final 15% to testing. This prevents look-ahead bias and produces honest model evaluation metrics.

9.5 Dashboard as a Single Aggregated API

Rather than having the frontend make 6 separate API calls on page load, the `/api/dashboard/summary` endpoint aggregates portfolio, alerts, market pulse, sparklines, and risk data in a single server-side request. concurrent I/O is used via `asyncio.gather()` for the price and candle fetches, keeping latency low.

10. Recruiter Value Proposition

Bullseye demonstrates the full engineering spectrum a senior developer or ML engineer is expected to own: system design, async programming, real-time data, financial domain modeling, ML pipeline construction, AI integration, debugging complex dependency chains, and user-facing product thinking.

Skill Area	Evidence in Bullseye
Full-Stack Architecture	React + TypeScript frontend, FastAPI async backend, SQLAlchemy ORM, WebSocket streaming — all integrated and working together
Real-Time Systems	WebSocket server/client with live LTP streaming, fallback logic, connection lifecycle management
ML Engineering	End-to-end pipeline: data fetching, feature engineering (40+ features), time-series-aware train/test split, XGBoost + LSTM dual-model architecture, auto-training, model caching, scaler persistence
AI Integration	Google Gemini with Google Search Retrieval grounding, Sentence Transformer embeddings (RAG-ready), VADER NLP — three distinct AI/ML paradigms in one project
Financial Domain Knowledge	NSE instrument resolution, Indian market hours detection, sector classification, technical indicators (RSI, EMA, MACD, ATR), risk metrics (Sharpe, Beta, VaR, HHI), portfolio analytics
API Design	25+ REST endpoints with proper HTTP semantics, Pydantic validation, JWT auth middleware, OpenAPI docs generation
Debugging & Problem Solving	Documented and solved 20+ production-grade issues across ML, async, WebSocket, auth, CORS, and broker API layers (see Section 8)
Production Thinking	Retry logic with exponential backoff (Upstox provider), graceful degradation when live prices unavailable, startup instrument registry caching, environment-based config via pydantic-settings
Product Thinking	India-first design (INR currency, NSE symbols, Indian market hours), user-friendly onboarding, drag-and-drop CSV import, quick prompt chips in AI chat
Code Organization	Clear separation: api/v1/, crud/, services/, models/, schemas.py — each layer has a single responsibility

11. Future Roadmap

11.1 Near-Term Technical Improvements

- **Train a single XGBoost model across all symbols using normalized features, reducing artifact storage from $O(\text{symbols})$ to $O(1)$ and enabling better generalization.** Generalized Multi-Symbol Model:
- **Replace SQLite with PostgreSQL for concurrent write safety, better query performance, and production scalability. The async SQLAlchemy setup requires only a connection string change.** PostgreSQL Migration:
- **Cache live price quotes and candle responses in Redis with a 1–5 second TTL to reduce Upstox API calls during high-traffic periods.** Redis Caching:
- **Move auto-training out of the request lifecycle into a Celery worker. This prevents the first prediction request from timing out for new symbols.** Background Training Jobs:
- **The current WebSocket endpoint skips JWT verification for stability. Add token-based handshake authentication as a production requirement.** WebSocket Authentication:

11.2 Product Feature Suggestions

- **Let users run prediction models against historical data with configurable entry/exit rules and compute strategy P/L, win rate, and Sharpe ratio.** Backtesting Engine:
- **Allow users to save symbols to a personal watchlist and set threshold alerts without navigating to the Alerts page.** Watchlist with Custom Alerts:
- **When concentration (HHI) exceeds 0.25, suggest which positions to trim and which underweight sectors to increase exposure to.** Portfolio Rebalancing Suggestions:
- **Abstract the Upstox provider to also support Zerodha Kite API, enabling users to connect their actual brokerage account for live P/L tracking.** Multi-Broker Support:
- **Integrate Firebase Cloud Messaging or WebPush to deliver triggered alerts to mobile browsers even when the tab is closed.** Push Notifications:
- **Add SHAP value visualizations for XGBoost predictions — show which features most influenced the UP/DOWN classification for each symbol.** Explainability Dashboard:

11.3 ML Improvements

- **Experiment with lightweight time-series Transformers (e.g., Temporal Fusion Transformer) to capture long-range dependencies better than LSTM.** Transformer-Based Models:
- **Feed VADER compound scores from the news module as an additional ML feature for same-day prediction tasks.** Sentiment Feature Integration:
- **Replace the static train/test split with rolling walk-forward validation to produce more reliable out-of-sample accuracy estimates.** Walk-Forward Validation:
- **Integrate Optuna for automated XGBoost hyperparameter tuning on a per-symbol**

basis during the auto-training pipeline. Hyperparameter Optimization:

12. Conclusion

Bullseye represents a complete, production-style fintech engineering project — not a tutorial application. It covers the full spectrum from real-time data ingestion and async API design, through to ML model pipelines, AI integration, and a polished user-facing product.

The project demonstrates domain competence in Indian equity markets, engineering discipline through documented problem-solving, and product maturity through thoughtful UX decisions such as India-first defaults, graceful degradation, and one-click AI explanations.

For a recruiter, Bullseye communicates: I can design systems, build end-to-end products, debug production problems, apply ML in realistic settings, integrate modern AI APIs, and deliver a feature-complete application with engineering discipline.

End of Report